



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Sistemas de la información

**Diseño de un sistema de telemetría para el análisis,
visualización y ayuda a la toma de decisiones
aplicado al deporte de motor y alta competicións**

**Design of a telemetry system for analysis,
visualization and decision support applied to
high-performance motorsport**

Realizado por

Darío Alessandro Di Totaro Ruíz

Tutorizado por

Francisco Javier Ferrer Urbano

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, Mayo, 2026

Resumen

El trabajo centra sus esfuerzos en el desarrollo de una herramienta gratuita de código abierto que permita a los usuarios analizar los múltiples aspectos fundamentales que influyen en el deporte de motor. Dada la amplitud de las diferentes categorías dentro de esta disciplina nos centraremos en la categoría más influyente, la Fórmula 1. Mediante un *dashboard* el usuario obtiene una visualización detallada de los pilotos y escuderías presentes en el transcurso de un gran premio, facilitando un análisis sencillo de cada carrera.

El principal objetivo del proyecto es proporcionar una herramienta intuitiva para personas con un conocimiento base del deporte, personas con un interés por el *simracing* o usuarios que quieran introducirse a la modalidad de una forma más técnica. Los usuarios pueden interactuar con los diferentes apartados y analizar detalles que puedan ayudar a mejorar su conocimiento sobre el deporte o comportamiento de los monoplazas analizando resultados, ritmo de carrera, distintas estrategias, fuerzas G del trazado o estrés del neumático. El desarrollo se ha llevado a cabo en base a Streamlit, un *framework* de Python de código abierto que permite construir aplicaciones web para el análisis de datos con capacidad para integrar gráficos. Python se ha utilizado como lenguaje aprovechando bibliotecas como Numpy, Pandas o Plotly para los distintos procesos que conlleva el manejo de datos. Se obtienen datos del monoplaza mediante el uso de APIs públicas como Ergast y la biblioteca FastF1 para la recolección de datos de la API de la Fórmula 1 así como una caché local para mejorar los tiempos de carga.

Se ha identificado como área de posible expansión la integración de datos de otras competiciones con mayor accesibilidad de datos, como el WEC (*World Endurance Championship*) o categorías de Fórmula inferiores.

Palabras clave: Fórmula 1, Análisis de datos, Python, Telemetría, Streamlit, Dashboard, Deportes

Abstract

We present a deep-learning pipeline for **MRI super-resolution** and **tumor segmentation**. A 3D U-Net segments lesions, while an EDSR-style SR module reconstructs high-resolution volumes. We optimize a composite loss ($L1 + SSIM + TV$) and report PSNR, SSIM, and Dice on brain MRI. Compared to classical interpolation and Random Forest baselines, the model reduces undersampling artifacts and sharpens clinically relevant boundaries.

Keywords: biomedical imaging, U-Net, super-resolution, segmentation, PSNR, SSIM, Dice

Agradecimientos

Agradezco su función a los clínicos colaboradores, así como al grupo de Inteligencia Computacional y Análisis de Imagen de la Universidad de Málaga.

Índice General

Resumen	1
Abstract	3
Agradecimientos	5
Índice General	7
Índice de Figuras	9
Índice de Tablas	11
1 Introduction	13
2 Fundamentals	15
2.1 Imaging model and objectives	15
2.2 Metrics	15
3 Methods	17
3.1 Network design	17
3.2 Mathematical foundations	18
3.3 Reference implementation (PyTorch)	19
4 Results	23
4.1 Quantitative metrics	23
4.2 Qualitative examples	23
5 Discussion	25
Apéndice A. Data splits and preprocessing	27
A.1 Preprocessing	27
Apéndice B. Additional code snippets	29
B.1 Evaluation script (Python)	29
B.2 R quick-look plot	29
B.3 C++ performance-critical kernels	30
B.4 SQL data pipeline	31
B.5 Bash automation	32

Índice de Figuras

1.1	Pipeline: LR MRI $\rightarrow$ SR reconstruction $\rightarrow$ segmentation. Training uses multi-task losses and shared augmentations.	13
4.1	Axial slices: LR input, SR output, and segmentation overlay.	23

Índice de Tablas

4.1	SR and segmentation performance (mean $\pm$ std).	23
-----	---	----

1

Introduction

Deep neural networks improved biomedical image reconstruction and analysis. We target MRI super-resolution and tumor segmentation in a unified workflow (Fig. 1.1).

[Pipeline diagram: LR MRI → SR reconstruction → segmentation]

Figura 1.1. Pipeline: LR MRI → SR reconstruction → segmentation. Training uses multi-task losses and shared augmentations.

2

Fundamentals

2.1 Imaging model and objectives

Assume $y = Ax + n$. The SR network f_θ estimates $\hat{x} = f_\theta(y)$. We use the composite objective

$$\mathcal{L}_{\text{SR}} = \lambda_1 \|\hat{x} - x\|_1 + \lambda_s (1 - \text{SSIM}(\hat{x}, x)) + \lambda_{\text{tv}} \text{TV}(\hat{x}). \quad (2.1)$$

For segmentation logits $z = g_\phi(\hat{x})$ and mask m ,

$$\mathcal{L}_{\text{seg}} = 1 - \text{Dice}(\sigma(z), m) + \beta \text{CE}(\sigma(z), m). \quad (2.2)$$

Total loss: $\mathcal{L} = \mathcal{L}_{\text{SR}} + \gamma \mathcal{L}_{\text{seg}}$.

2.2 Metrics

Performance metrics

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{L^2}{\text{MSE}(x, \hat{x})}, \quad (2.3)$$

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x \mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}, \quad (2.4)$$

$$\text{Dice}(p, m) = \frac{2 \sum pm}{\sum p + \sum m}. \quad (2.5)$$

3

Methods

3.1 Network design

SR module: residual blocks with sub-pixel upsampling. Segmentation module: 3D U-Net with skip connections. [1]

Algorithm 1 Joint SR+Seg training

```
1: procedure TRAIN( $\mathcal{D}, \theta, \phi$ )
2:   for epoch = 1.. $E$  do
3:     for batch  $(y, x, m) \sim \mathcal{D}$  do
4:        $\hat{x} \leftarrow f_{\theta}(y)$ 
5:        $z \leftarrow g_{\phi}(\hat{x}); p \leftarrow \sigma(z)$ 
6:        $\mathcal{L} \leftarrow \mathcal{L}_{\text{SR}}(\hat{x}, x) + \gamma \mathcal{L}_{\text{seg}}(p, m)$ 
7:       Update  $(\theta, \phi)$  by Adam on  $\nabla \mathcal{L}$ 
8:     end for
9:   end for
10: end procedure
```

3.2 Mathematical foundations

Definición 1 (Structural Similarity Index)

The **Structural Similarity Index (SSIM)** between two images x and $\hat{x}$ is defined as:

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

where $\mu_x, \mu_{\hat{x}}$ are the local means, $\sigma_x^2, \sigma_{\hat{x}}^2$ are the local variances, and $\sigma_{x\hat{x}}$ is the cross-covariance.

Teorema 2 (Convergence of Gradient Descent)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a convex, L -smooth function. Then gradient descent with step size $\eta = 1/L$ satisfies:

$$f(x_k) - f(x^*) \leq \frac{L\|x_0 - x^*\|^2}{2k}$$

where x^* is the global minimizer.

Demostración. By L -smoothness, we have $f(y) \leq f(x) + \nabla f(x)^\top(y - x) + \frac{L}{2}\|y - x\|^2$. Setting $y = x - \frac{1}{L}\nabla f(x)$ and using convexity, the result follows by telescoping. $\square$

Lema 3 (Dice Loss Gradient)

The gradient of the Dice loss with respect to the predicted probability p_i is:

$$\frac{\partial \mathcal{L}_{\text{Dice}}}{\partial p_i} = -\frac{2m_i(\sum_j p_j + \sum_j m_j) - 2\sum_j p_j m_j}{(\sum_j p_j + \sum_j m_j)^2}$$

Corolario 4 (Optimal Learning Rate)

For neural networks with batch normalization, the optimal learning rate scales as $\eta \propto \sqrt{B}$ where B is the batch size.

Proposición 5 (Universal Approximation)

A feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on compact subsets of $\mathbb{R}^n$, under mild assumptions on the activation function.

Observación

The convergence rate in Teorema 2 is $O(1/k)$, which can be improved to $O(1/k^2)$ using accelerated methods like Nesterov momentum.

Ejemplo

Consider a 3D U-Net with depth $D = 4$ and initial channels $C = 32$. The receptive field grows exponentially with depth, allowing the network to capture both local texture and global context.

Nota

All experiments were conducted on NVIDIA A100 GPUs with mixed-precision training enabled.

3.3 Reference implementation (PyTorch)

```
1  # Module: sr_seg.py
2  # Purpose: Joint SR reconstruction and segmentation for MRI volumes
3  # Notes: Typed, logged, dataclass configs, minimal structure
4
5  from dataclasses import dataclass
6  from typing import Tuple
7  import logging, torch, torch.nn as nn
8  import torch.nn.functional as F
9
10 logger = logging.getLogger("srseg")
11 logger.setLevel(logging.INFO)
12
13 @dataclass
14 class TrainConfig:
15     lr: float = 2e-4
16     gamma: float = 1.0
17     lambda_l1: float = 1.0
18     lambda_ssim: float = 0.2
19     lambda_tv: float = 1e-5
20
21 class ResidualBlock(nn.Module):
22     """Residual block with two conv3d layers."""
23     def __init__(self, c: int):
24         super().__init__()
25         self.c1 = nn.Conv3d(c, c, 3, padding=1)
```

```

26         self.c2 = nn.Conv3d(c, c, 3, padding=1)
27     def forward(self, x: torch.Tensor) ->torch.Tensor:
28         r = F.relu(self.c1(x))
29         r = self.c2(r)
30         return F.relu(x + r)
31
32     class SRNet(nn.Module):
33         """EDSR-like SR backbone for volumetric data."""
34         def __init__(self, in_ch: int = 1, base: int = 64, depth: int = 8):
35             super().__init__()
36             self.head = nn.Conv3d(in_ch, base, 3, padding=1)
37             self.body = nn.Sequential(*[ResidualBlock(base) for _ in range(depth
38                                     )])
39             self.tail = nn.Conv3d(base, 1, 3, padding=1)
40         def forward(self, y: torch.Tensor) ->torch.Tensor:
41             h = self.head(y)
42             h = self.body(h)
43             return self.tail(h)
44
45     class UNet3D(nn.Module):
46         """3D U-Net for segmentation."""
47         def __init__(self, in_ch: int = 1, n_classes: int = 2, base: int = 32):
48             super().__init__()
49             self.e1 = nn.Sequential(nn.Conv3d(in_ch, base, 3, padding=1), nn.
50                                     ReLU(),
51                                     nn.Conv3d(base, base, 3, padding=1), nn.ReLU())
52             self.e2 = nn.Sequential(nn.Conv3d(base, base*2, 3, stride=2, padding
53                                     =1), nn.ReLU())
54             self.b = nn.Sequential(nn.Conv3d(base*2, base*4, 3, padding=1), nn.
55                                     ReLU())
56             self.d2 = nn.Sequential(nn.ConvTranspose3d(base*4, base*2, 2, stride
57                                     =2), nn.ReLU())
58             self.out= nn.Conv3d(base*2, n_classes, 1)
59         def forward(self, x: torch.Tensor) ->torch.Tensor:

```

```

55     x1 = self.e1(x)
56     x2 = self.e2(x1)
57     b = self.b(x2)
58     d2 = self.d2(b)
59     z = self.out(d2)
60     return z
61
62 def dice_loss(p: torch.Tensor, m: torch.Tensor, eps: float = 1e-6) ->torch.
    Tensor:
63     p = p.softmax(1)[:,:1] # foreground
64     m = (m > 0.5).float()
65     num = 2*(p*m).sum()
66     den = p.sum() + m.sum() + eps
67     return 1 - num/den
68
69 def tv(x: torch.Tensor) ->torch.Tensor:
70     return (x[:, :, 1:] - x[:, :, :-1]).abs().mean() + \
71           (x[:, :, :, 1:] - x[:, :, :, :-1]).abs().mean() + \
72           (x[:, :, :, :, 1:] - x[:, :, :, :, :-1]).abs().mean()
73
74 def train_step(sr: SRNet, unet: UNet3D, opt: torch.optim.Optimizer,
75               y: torch.Tensor, x: torch.Tensor, m: torch.Tensor,
76               cfg: TrainConfig) ->float:
77     opt.zero_grad()
78     x_hat = sr(y)
79     l1 = (x_hat - x).abs().mean()
80     # crude SSIM proxy to keep example short
81     ssim_term = ((x_hat - x)**2).mean()
82     l_tv = tv(x_hat)
83     z = unet(x_hat.detach())
84     ce = F.cross_entropy(z, m.long())
85     dl = dice_loss(z, m)
86     loss = cfg.lambda_l1*l1 + cfg.lambda_ssim*ssim_term + cfg.lambda_tv*l_tv
        + cfg.gamma*(ce + dl)

```

```
87     loss.backward()
88     opt.step()
89     logger.info(f"loss={loss.item():.4f}")
90     return float(loss.item())
```

Listing 3.1. *Minimal SR+U_Net skeleton (PyTorch)*

4

Results

4.1 Quantitative metrics

Tabla 4.1. SR and segmentation performance (mean $\pm$ std).

Method	PSNR [dB]	SSIM	Dice
Bicubic + RF	28,31	0,842	0,786
EDSR + U ³ Net	31,75	0,904	0,865
Proposed (joint)	32,18	0,918	0,882

4.2 Qualitative examples

[Axial slices: LR input, SR output, and segmentation overlay]

Figura 4.1. Axial slices: LR input, SR output, and segmentation overlay.

5

Discussion

Joint optimization improves boundary fidelity and suppresses ringing. Failure cases include motion artifacts and unseen acquisition protocols.

Apéndice A. Data splits and preprocessing

A.1 Preprocessing

Bias-field correction, z-score normalization per volume, random flips and intensity jitter for augmentation.

Apéndice B. Additional code snippets

B.1 Evaluation script (Python)

```
1  # eval_metrics.py
2  # Computes PSNR, SSIM (skimage), and Dice given predictions and references.
3
4  from skimage.metrics import structural_similarity as ssim
5  import numpy as np
6
7  def psnr(x, y, peak=1.0):
8      mse = np.mean((x - y) ** 2)
9      return 10 * np.log10(peak**2 / (mse + 1e-12))
10
11 def dice(p, m, thr=0.5):
12     pbin = (p >= thr).astype(np.float32)
13     m = m.astype(np.float32)
14     return 2*(pbin*m).sum() / (pbin.sum() + m.sum() + 1e-12)
```

Listing 5.1. Metric computation script

B.2 R quick-look plot

```
1  # rplots.R - Statistical visualization of model performance
2  library(ggplot2)
3  library(dplyr)
4
```

```

5 # Generate synthetic results for demonstration
6 df <-data.frame(
7   method = rep(c("Bicubic+RF", "EDSR+U3Net", "Joint"), each=10),
8   psnr = c(rnorm(10,28.3,0.6), rnorm(10,31.8,0.5), rnorm(10,32.2,0.4)),
9   dice = c(rnorm(10,0.786,0.02), rnorm(10,0.865,0.02), rnorm(10,0.882,0.02))
10 )
11
12 # Create visualization pipeline
13 df %>%
14   ggplot(aes(method, psnr, fill=method)) +
15   geom_boxplot() +
16   theme_minimal() +
17   labs(title="PSNR by Method", y="PSNR (dB)")

```

Listing 5.2. Boxplots of PSNR and Dice

B.3 C++ performance-critical kernels

```

1 // conv3d_kernel.cpp - High-performance 3D convolution
2 #include <vector>
3 #include <cmath>
4 #include <algorithm>
5
6 template<typename T>
7 class Conv3DKernel {
8 private:
9   std::vector<T> weights_;
10   int kernel_size_;
11
12 public:
13   Conv3DKernel(int size) : kernel_size_(size) {
14     weights_.resize(size * size * size);

```



```

15     }
16
17     // Apply convolution at a single point
18     T apply(const T* input, int x, int y, int z, int stride) const {
19         T sum = static_cast<T>(0);
20         for (int dz = 0; dz < kernel_size_; ++dz) {
21             for (int dy = 0; dy < kernel_size_; ++dy) {
22                 for (int dx = 0; dx < kernel_size_; ++dx) {
23                     int idx = (z + dz) * stride * stride
24                             + (y + dy) * stride + (x + dx);
25                     sum += input[idx] * weights_[dz * kernel_size_ *
26                                                kernel_size_
27                                                + dy * kernel_size_ + dx];
28                 }
29             }
30         }
31         return sum;
32     };

```

Listing 5.3. CUDA-style convolution kernel

B.4 SQL data pipeline

```

1  -- cohort_selection.sql
2  -- Select patients with complete MRI sequences for training
3  SELECT DISTINCT
4      p.patient_id,
5      p.age,
6      p.diagnosis_date,
7      COUNT(s.scan_id) AS num_scans
8  FROM patients p
9  INNER JOIN mri_scans s ON p.patient_id = s.patient_id

```

```

10 WHERE s.modality = 'T1w'
11     AND s.quality_score >= 0.8
12     AND p.diagnosis_date BETWEEN '2020-01-01' AND '2024-12-31'
13 GROUP BY p.patient_id, p.age, p.diagnosis_date
14 HAVING COUNT(s.scan_id) >= 3
15 ORDER BY p.diagnosis_date DESC
16 LIMIT 1000;

```

Listing 5.4. Patient cohort selection query

B.5 Bash automation

```

1  #!/bin/bash
2  # train_pipeline.sh - Automated model training and evaluation
3
4  # Configuration
5  MODEL_DIR="./checkpoints"
6  DATA_DIR="/data/mri_volumes"
7  EPOCHS=100
8
9  # Create output directories
10 mkdir -p "$MODEL_DIR" logs
11
12 # Train with different learning rates
13 for lr in 1e-3 1e-4 1e-5; do
14     echo "Training with lr=$lr"
15     python train.py \
16         --data "$DATA_DIR" \
17         --lr "$lr" \
18         --epochs "$EPOCHS" \
19         --output "$MODEL_DIR/model_lr${lr}.pt" \
20         2>&1 | tee "logs/train_lr${lr}.log"
21 done

```

```
22
23 # Evaluate best model
24 echo "Evaluating models..."
25 python evaluate.py --checkpoint "$MODEL_DIR" --metrics psnr ssim dice
```

Listing 5.5. *Training pipeline automation*

Bibliografía

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *nature* 521.7553 (2015), pp. 436–444.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga